

计算机体系结构

新型计算机研究所

陈 衡

西一楼 A 段415室

Email: hengchen@xjtu.edu.cn

QQ群号: 1031141115

第4章 向量处理机

- ◆ 4.1 向量的处理方式
- ◆ 4.2 向量处理机的结构
- ◆ 4.3 提高向量处理机性能的常用技术
- ◆ 4.4 向量处理机的性能评价
- ◆ 4.5 向量处理机实例

第4章 向量处理机

◆ 概念

- 在流水线处理机中，设置向量数据表示和相应的向量指令，称为**向量处理机**。
- 不具有向量数据表示和相应的向量指令的流水线处理机，称为**标量处理机**。

◆ 4.1 向量处理方式

- 以计算表达式 $D = A \times (B + C)$ 为例
- A、B、C、D — 长度为N的向量
- 处理方式
 - 横向(水平)处理方式
 - 纵向(垂直)处理方式
 - 纵横(分组)处理方式

4.1 向量处理方式

1. 横向(水平)处理方式

➤ 向量计算是按行的方式从左到右横向地进行。

□ 先计算: $d_1 \leftarrow a_1 \times (b_1 + c_1)$

□ 再计算: $d_2 \leftarrow a_2 \times (b_2 + c_2)$

□

□ 最后计算: $d_N \leftarrow a_N \times (b_N + c_N)$

➤ 组成循环程序进行处理。

$$k_i \leftarrow b_i + c_i$$

$$d_i \leftarrow k_i \times a_i$$

□ 数据相关: N 次 功能切换: $2N$ 次

➤ 不适合于向量处理机的并行处理。

4.1 向量处理方式

2. 纵向 (垂直)处理方式

- 向量计算是按列的方式从上到下纵向地进行。

	$k_1 \leftarrow b_1 + c_1$		$d_1 \leftarrow k_1 \times a_1$
先计算		再计算	
	$k_N \leftarrow b_N + c_N$		$d_N \leftarrow k_N \times a_N$

- 表示成向量指令：

$$K = B + C$$

$$D = K \times A$$

- 两条向量指令之间：

数据相关：1次 功能切换：1次

4.1 向量处理方式

3. 纵横 (分组)处理方式

- 又称为**分组处理方式**。
- 把向量分成若干组，组内按纵向方式处理，依次处理各组。
- 对于上述的例子，设：

$$N = S \times n + r$$

- 其中 N 为向量长度， S 为组数， n 为每组的长度， r 为余数。
- 若余下的 r 个数也作为一组处理，则共有 $S+1$ 组。
- 运算过程为：

4.1 向量处理方式

- 先算第1组:

$$k_{1\sim n} \leftarrow b_{1\sim n} + c_{1\sim n}$$

$$d_{1\sim n} \leftarrow k_{1\sim n} \times a_{1\sim n}$$

- 再算第2组:

$$k_{(n+1)\sim 2n} \leftarrow b_{(n+1)\sim 2n} + c_{(n+1)\sim 2n}$$

$$d_{(n+1)\sim 2n} \leftarrow k_{(n+1)\sim 2n} \times a_{(n+1)\sim 2n}$$

- 依次进行下去, 直到最后一组: 第 $S+1$ 组。
- 每组内各用两条向量指令。

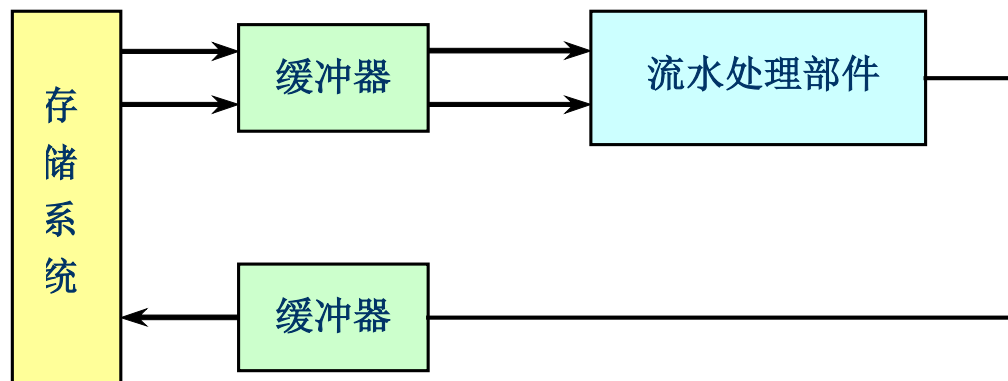
数据相关: 1次 功能切换: 2次

4.2 向量处理机的结构

- ◆ 向量处理机的结构因具体机器不同而不同
 - 由所采用的向量处理方式决定
- ◆ 有两种典型的结构
 - 存储器-存储器型结构
 - 纵向处理方式采用
 - 寄存器-寄存器型结构
 - 分组处理方式采用

4.2.1 “存储器-存储器”结构

- ◆ 采用纵向处理方式的向量处理机对处理机结构的要求：存储器—存储器结构
- ◆ 向量指令的源向量和目的向量都是存放在存储器中，运算的中间结果需要送回存储器。
- ◆ 流水线运算部件的输入和输出端都直接（或经过缓冲器）与存储器相联，从而构成存储器—存储器型操作的运算流水线。
 - 例如：STAR-100、CYBER-205



4.2.1 “存储器-存储器”结构

- ◆ 要充分发挥这种结构的流水线效率，存储器要不断地提供源操作数，并不断地从运算部件接收结果。
 - 每拍从存储器读取两个数据，并向存储器写回一个结果
- ◆ 对存储器的带宽以及存储器与处理部件的通信带宽提出了非常高的要求。
- ◆ 解决方法：一般是通过采用多体交叉并行存储器和缓冲器技术。
 - 例如，70年代初问世的Star 100
 - 存储器：32个体交叉
 - 每个体的数据宽度：8个字（字长64位）
 - 最大数据流量：每秒2亿字

4.4.2 “寄存器-寄存器”结构

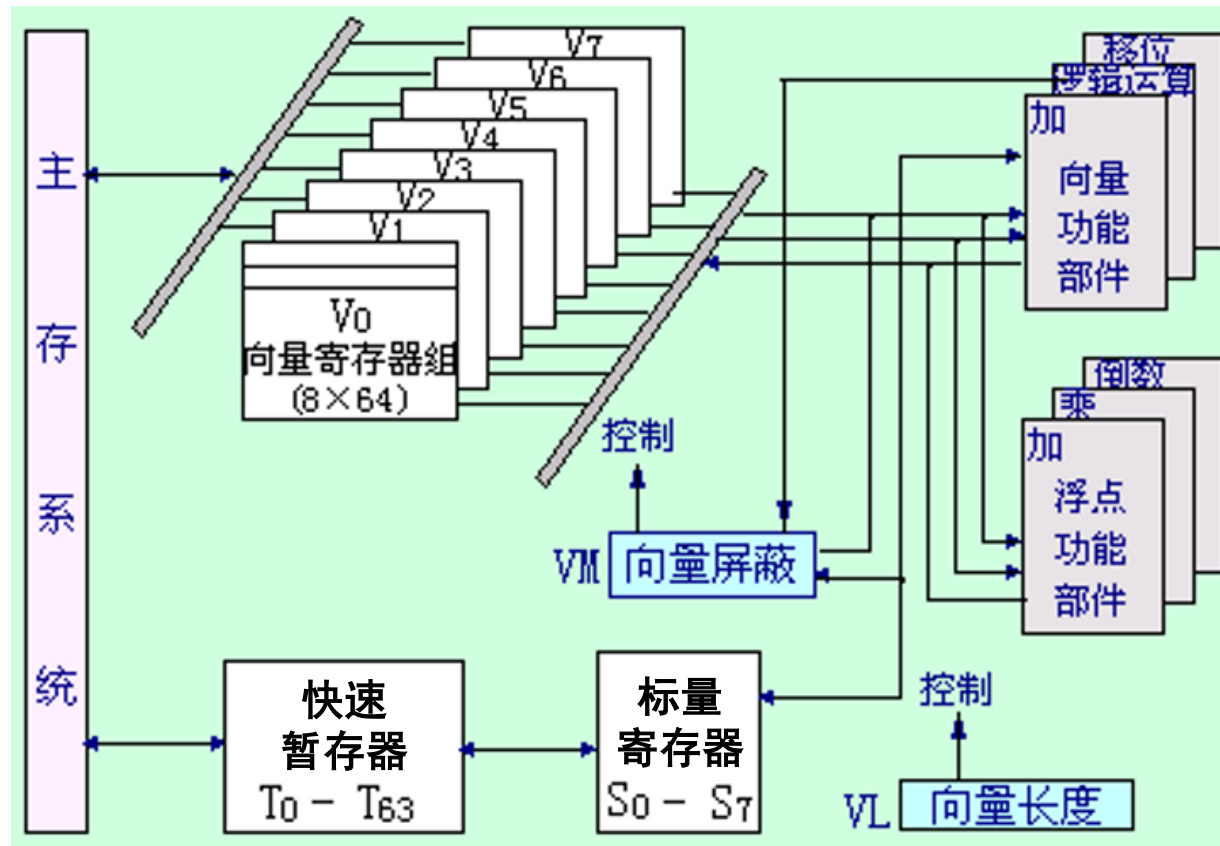
- ◆ 在向量的分组处理方式中，对向量长度 N 没有限制，但组的长度 n 却是固定不变的。
- ◆ 对处理机结构的要求：寄存器—寄存器结构
- ◆ 设置能快速访问的**向量寄存器**，用于存放源向量、目的向量及中间结果。让运算部件的输入、输出端都与向量寄存器相联，就构成了“寄存器—寄存器”型操作的运算流水线。
- ◆ 典型的寄存器—寄存器结构的向量处理机
 - 美国的CRAY-1、我国的YH-1巨型机

4.4.2 “寄存器-寄存器”结构

◆ 以CRAY-1机为例

- 美国CRAY公司，1976年
- 每秒1亿次浮点运算，时钟周期：12.5 ns

• CRAY-1的基本结构



4.4.2 “寄存器-寄存器”结构

◆ CRAY-1的基本结构

◆ 1、功能部件

- 共有12条可并行工作的单功能流水线，可分别流水地进行地址、向量、标量的各种运算。
- 6个单功能流水部件：进行向量运算
 - 整数加（3拍）
 - 逻辑运算（2拍）
 - 移位（4拍）
 - 浮点加（6拍）
 - 浮点乘（7拍）
 - 浮点迭代求倒数（14拍）
 - 括号中的数字为其流水经过的时间，每拍为一个时钟周期，即12.5 ns。

4.4.2 “寄存器-寄存器”结构

◆ 2、向量寄存器V

- 由512个64位的寄存器组成，分成8块。编号： $V_0 \sim V_7$
- 每一个块称为一个向量寄存器，可存放一个长度（即元素个数）不超过64的向量。
- 每个向量寄存器可以每拍向功能部件提供一个数据元素，或者每拍接收一个从功能部件来的结果元素。

◆ 3、标量寄存器S和快速暂存器T

- 标量寄存器有8个： $S_0 \sim S_7$ 64位
- 快速暂存器T用于在标量寄存器和存储器之间提供缓冲。

◆ 4、向量屏蔽寄存器VM

- 64位，每一位对应于向量寄存器的一个单元。
- 作用：用于向量的归并、压缩、还原和测试操作、对向量某些元素的单独运算等。

4.4.2 “寄存器-寄存器”结构

◆ CRAY-1向量处理的一个显著特点

- 每个向量寄存器 V_i 都有连到6个向量功能部件的单独总线
- 每个向量功能部件有把运算结果送回向量寄存器组的总线
- 只要不出现 V_i 冲突和功能部件冲突，各 V_i 之间和各功能部件之间都能并行工作，大大加快了向量指令的处理。

■ **V_i 冲突**：并行工作的各向量指令的源向量或结果向量使用相同的 V_i

■ 例如：源向量相同

$$V_3 \leftarrow V_1 + V_2$$

$$V_5 \leftarrow V_4 \wedge V_1$$

■ **功能部件冲突**：并行工作的各向量指令要使用同一个功能部件。

■ 例如：都需使用乘法功能部件

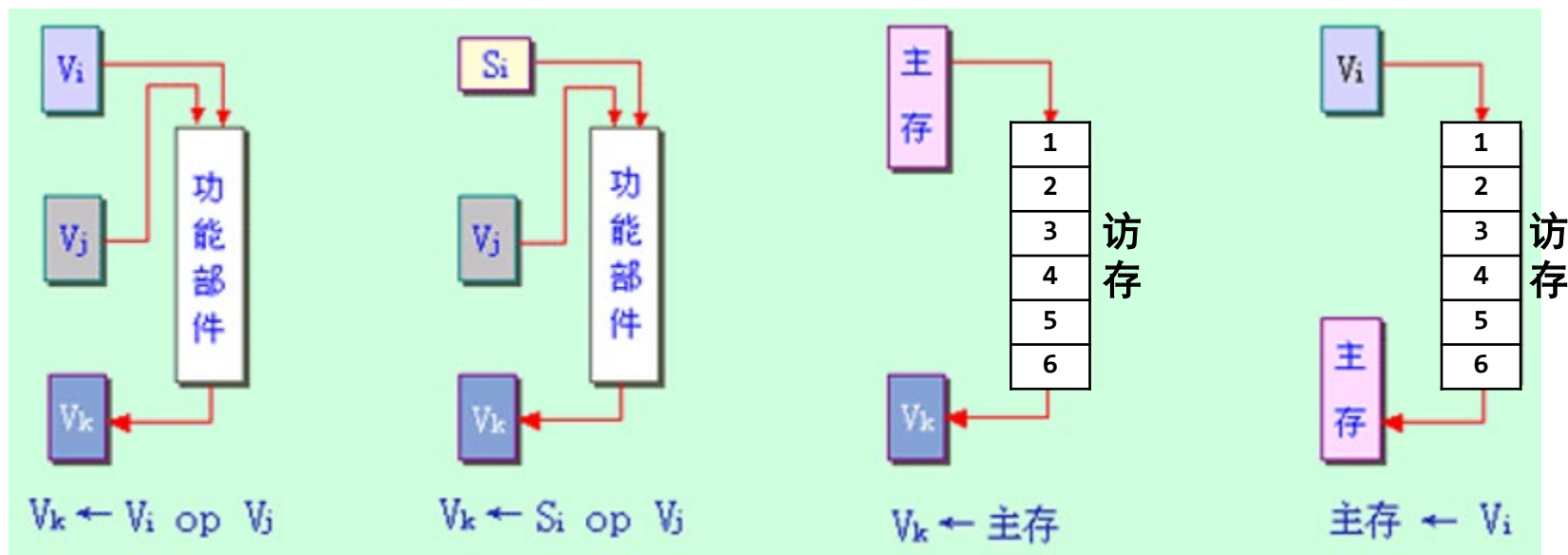
$$V_3 \leftarrow V_1 \times V_2$$

$$V_5 \leftarrow V_4 \times V_6$$

4.4.2 “寄存器-寄存器”结构

◆ CRAY-1向量指令类型

- $V_k \leftarrow V_i \text{ op } V_j$
- $V_k \leftarrow S_i \text{ op } V_j$
- $V_k \leftarrow \text{主存}$
- $\text{主存} \leftarrow V_i$



4.3 提高向量处理机性能的方法

◆ 提高向量处理机性能的方法

- 设置多个功能部件，使它们并行工作。
- 采用链接技术，加快一串向量指令的执行。
- 采用循环开采技术，加快循环的处理。
- 采用多处理机系统，进一步提高性能。

◆ 4.3.1 设置多个功能部件

- 设置多个独立的功能部件。它们能并行工作，并各自按流水方式工作，从而形成了**多条并行工作**的运算操作流水线
- 例如：CRAY-1向量处理机有4组12个单功能流水部件：
 - 向量部件：向量加，移位，逻辑运算
 - 浮点部件：浮点加，浮点乘，浮点求倒数
 - 标量部件：标量加，移位，逻辑运算，数“1”/计数
 - 地址运算部件：整数加，整数乘

4.3.2 链接技术

◆ 链接技术

- 链接特征：具有先写后读相关的两条指令，在不出现功能部件冲突和源向量冲突的情况下，可以把功能部件链接起来进行流水处理，以达到加快执行的目的。
- 链接特性的实质
 - 把**流水线定向的思想**引入到向量执行过程的结果。

◆ 举例：

- 在CRAY-1上用链接技术进行向量运算 $D=A \times (B+C)$
- 假设向量长度 $N \leq 64$ ，向量元素为浮点数，且向量B、C已存放在V0和V1中。
- 画出链接示意图，并分析非链接执行和链接执行两种情况下的执行时间。

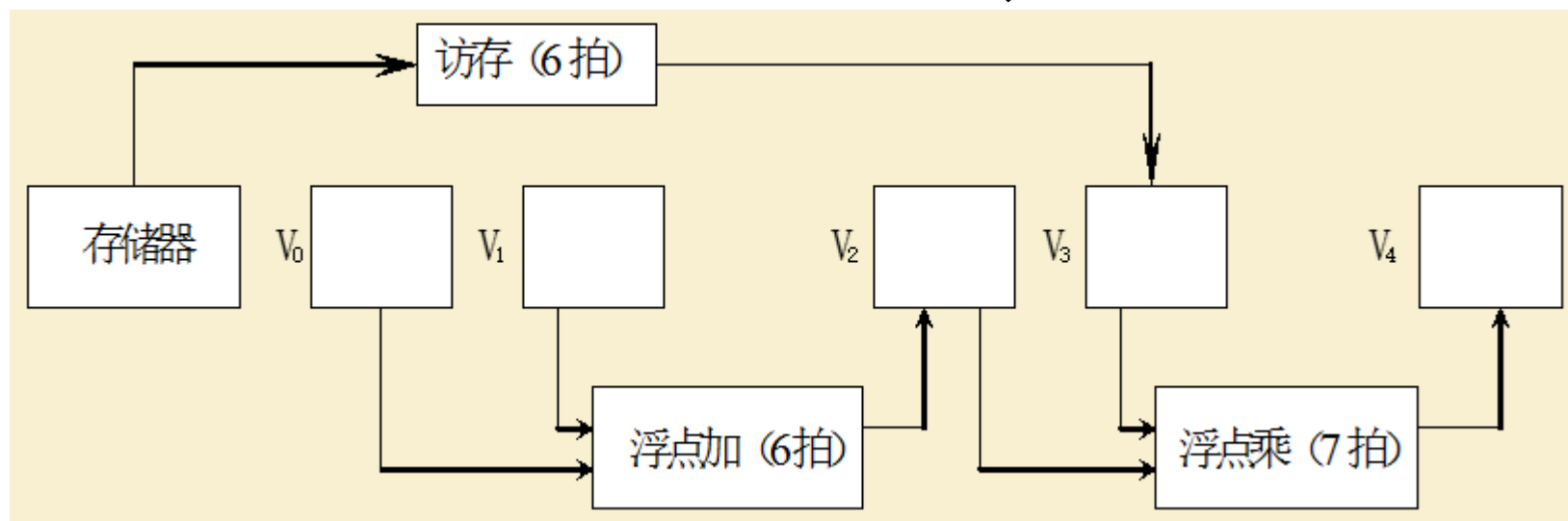
4.3.2 链接技术

◆ 解 用以下三条向量完成上述运算：

$V_3 \leftarrow \text{存储器}$ // 访存取向量A

$V_2 \leftarrow V_0 + V_1$ // 向量B和向量C进行浮点加

$V_4 \leftarrow V_2 \times V_3$ // 浮点乘，结果存入V4



- **假设：**把向量数据元素送往向量功能部件以及把结果存入向量寄存器需要**一拍时间**，从存储器中把数据送入访存功能部件需要**一拍时间**。

4.3.2 链接技术

- ◆ (1) 3条指令全部用串行方法执行，则执行时间为：
$$[(1+6+1) + N - 1] + [(1+6+1) + N - 1] + [(1+7+1) + N - 1] = 3N + 22 \text{ (拍)}$$
- ◆ (2) 前两条指令并行执行，然后再串行执行第3条指令，则执行时间为：
$$[(1+6+1) + N - 1] + [(1+7+1) + N - 1] = 2N + 15 \text{ (拍)}$$
- ◆ (3) 第1、2条向量指令并行执行，并与第3条指令链接执行
- ◆ 从访存开始到把第一个结果元素存入V4所需的拍数
- ◆ (亦称为链接流水线的建立时间) 为：
$$[(1+6+1)] + [(1+7+1)] = 17 \text{ (拍)}$$
- ◆ 3条指令的执行时间为：
$$[(1+6+1)] + [(1+7+1)] + (N-1) = N + 16 \text{ (拍)}$$

4.3.2 链接技术

◆ 进行向量链接的要求

- 保证：无向量寄存器使用冲突和无功能部件使用冲突
- 只有在前一条指令的第一个结果元素送入结果向量寄存器的那一个时钟周期才可以进行链接。
- 当一条向量指令的两个源操作数分别是两条先行指令的结果寄存器时，要求**先行的两条指令产生运算结果的时间必须相等**，即要求有关功能部件的通过时间相等。
- 要进行链接执行的向量指令的向量长度必须相等，否则无法进行链接。

4.3.3 分段开采技术

◆ 分段开采技术

- 如果向量的长度大于向量寄存器的长度，该如何处理呢？
- 当向量的长度大于向量寄存器的长度时，必须把长向量分成长度固定的段，然后循环分段处理，每一次循环只处理一个向量段。
- 这种技术称为分段开采技术。
 - 由系统硬件和软件控制完成，对程序员是透明的。

◆ 举例

- 设A和B是长度为N的向量，考虑在Cray-1向量处理器上实现以下的循环操作：

DO 10 I = 1, N

10 A(I) = 5.0 * B(I) + C

4.3.3 分段开采技术

◆ 当 $N \leq 64$ 时，可以用以下指令序列：

$S1 \leftarrow 5.0$	； 将常数5.0送入标量寄存器S1
$S2 \leftarrow C$	； 将常数C送入标量寄存器S2
$VL \leftarrow N$	； 在向量长度寄存器VL中设置向量长度N
$V0 \leftarrow B$	； 从存储器中将向量B读入向量寄存器V0
$V1 \leftarrow S1 \times V0$	； 向量B中的每个元素分别和常数S1相乘
$V2 \leftarrow S2 + V1$	； 向量V1中的每个元素分别和常数S2相加
$A \leftarrow V2$	； 将计算结果从向量寄存器V2存入存储器
	； 的向量A

4.3.3 分段开采技术

◆ 当 $N > 64$ 时，就需要进行分段开采。

• 循环次数 K :
$$K = \left\lfloor \frac{N}{64} \right\rfloor$$

• 余数 L :
$$L = N - 64 \times \left\lfloor \frac{N}{64} \right\rfloor$$

$S1 \leftarrow 5.0$	； 将常数5.0送入标量寄存器S1
$S2 \leftarrow C$	； 将常数C送入标量寄存器S2
$VL \leftarrow L$	； 在向量长度寄存器VL中设置向量长度L(余数)
$V0 \leftarrow B$	； 从存储器中将向量B[0..L-1]读入向量寄存器V0
$V1 \leftarrow S1 \times V0$	； 向量B中的每个元素分别和常数S1相乘
$V2 \leftarrow S2 + V1$	； 向量V1中的每个元素分别和常数S2相加
$A \leftarrow V2$	； 将计算结果从向量寄存器V2存入存储器
	； 的向量A[0..L-1]

4.3.3 分段开采技术

```
For (I=0 to K-1) {  
    V0 ← B           ; 从存储器中将向量B[L+I*64..L+I*64+63]  
                     ; 读入向量寄存器V0  
    V1 ← S1 * V0      ; 向量B中的每个元素分别和常数S1相乘;  
    V2 ← S2 + V1      ; 向量V1中的每个元素分别和常数S2 相加  
    A ← V2           ; 将计算结果V2存入存储器的向量  
                     ; A[L+I*64... L+I*64+63]  
}
```

◆ 4.3.4 采用多处理机系统

- 许多新型向量处理机系统采用了多处理机系统结构。例如
- CRAY-2: 包含了4个向量处理机,
 - 浮点运算速度最高可达1800MFLOPS
- CRAY Y-MP、C90: 最多可包含16个向量处理机

4.4 向量处理机的性能评价

◆ 衡量向量处理机性能的主要参数：

- 向量指令的处理时间 T_{vp}
- 向量处理机的最大性能 R_{∞}
- 半性能向量长度 $n_{1/2}$
- 向量长度临界值 n_v

◆ 4.4.1 向量指令的处理时间

- 执行一条向量长度为 n 的向量指令所需的时间为

$$T_{vp} = T_s + T_{vf} + (n-1)T_c$$

- T_s ：向量流水线的建立时间，准备时间
- T_{vf} ：向量流水线的流过时间
 - 它是从向量指令开始译码算起，到第一对向量元素流过流水线直到产生第一个结果元素所需的时间。
- T_c ：流水线瓶颈段的执行时间

4.4.1 向量指令的处理时间

- ◆ 如果流水线不存在“瓶颈”，每段的执行时间等于一个时钟周期，则上式可以写为：

$$T_{vp} = [s + e + (n - 1)] T_{clk}$$

- s ：向量流水线的建立时间所对应的时钟周期数
- e ：向量流水线的流过时间所对应的时钟周期数
- T_{clk} ：时钟周期时间

- ◆ 也可以将上式改写为：

$$T_{vp} = [T_{start} + n] T_{clk}$$

- T_{start} ：向量功能部件启动所需的时钟周期数

4.4.1 向量指令的处理时间

- ◆ 对于一组向量指令而言，其执行时间主要取决于三个因素：
 - 向量的长度
 - 向量操作之间是否链接
 - 向量功能部件的冲突和数据的冲突性
- ◆ 把几条能在同一个时钟周期内一起开始执行的向量指令集合称为一个**编队**。
 - 可以看出，同一个编队中的向量指令之间一定不存在流水向量功能部件的冲突和数据的冲突。

4.4.1 向量指令的处理时间

◆ 举例：

例4.4 假设每种向量功能部件只有一个，那么下面的一组向量指令能分成几个编队？

LV V1, Rx

MULTSV V2, R0, V1

LV V3, Ry

ADDV V4, V2, V3

SV Ry, V4

◆ 解： 分为4个编队

- 第一编队： LV 第二编队： MULTSV; LV
- 第三编队： ADDV 第四编队： SV

4.4.1 向量指令的处理时间

- ◆ 一个编队内所有向量指令执行完毕所要的时间为：
- ◆ 编队后的向量指令序列总的执行时间为：

- 假设第*i*个编队中所有向量指令处理的向量元素个数均为*n*

$$\begin{aligned} T_c^i &= \max_j \left[T_{start}^{ij} + n \right] T_{clk} \\ &= \left(T_{start}^i + n \right) T_{clk} \end{aligned}$$

- T_c^i ：第*i*个编队的执行时间
- T_{start}^{ij} ：第*i*个编队中第*j*条指令所使用向量功能部件的启动时钟周期数

$$\begin{aligned} T_v &= \sum_{i=1}^m T_c^i \\ &= \sum_{i=1}^m \left(T_{start}^i + n \right) T_{clk} \\ &= \left(\sum_{i=1}^m T_{start}^i + mn \right) T_{clk} \\ &= \left(T_{start} + mn \right) T_{clk} \end{aligned}$$

- *m*：向量指令序列编队的个数
- T_{start} ：向量指令序列编队总的启动时钟周期数

4.4.1 向量指令的处理时间

- ◆ 若向量长度 n 大于向量处理机内部的向量寄存器长度
 - 额外开销 T_{loop}

$$\begin{aligned} T_n &= \left\lfloor \frac{n}{MVL} \right\rfloor \times (T_{loop} + T_{start} + m \times MVL) \\ &\quad + \left[T_{loop} + T_{start} + m \times \left(n - \left\lfloor \frac{n}{MVL} \right\rfloor \times MVL \right) \right] \\ &= \left\lfloor \frac{n}{MVL} \right\rfloor \times (T_{loop} + T_{start}) + mn \end{aligned}$$

4.4.1 向量指令的处理时间

◆ 举例

LV V1, Rx
 MULTSV V2, R0, V1
 LV V3, Ry
 ADDV V4, V2, V3
 SV Ry, V4

T_{loop}	15 cc
存储单元启动	12 cc
向量乘法部件启动	7 cc
向量加法部件启动	6 cc
向量寄存器长度	64

◆ 4个编队时

$$\begin{aligned}
 T_n &= \left\lceil \frac{n}{MVL} \right\rceil \times (T_{loop} + T_{start}) + mn \\
 &= \left\lceil \frac{n}{64} \right\rceil \times (15 + 12 + 12 + 6 + 12) + 4n \\
 &= \left\lceil \frac{n}{64} \right\rceil \times 57 + 4n
 \end{aligned}$$

使用链接, 3个编队

$$\begin{aligned}
 T_n &= \left\lceil \frac{n}{MVL} \right\rceil \times (T_{loop} + T_{start}) + mn \\
 &= \left\lceil \frac{n}{64} \right\rceil \times (15 + 19 + 18 + 12) + 3n \\
 &= \left\lceil \frac{n}{64} \right\rceil \times 64 + 3n
 \end{aligned}$$

4.4.2 最大性能和半性能向量长度

◆ 1、向量处理机的峰值性能 R_{∞}

- R_{∞} 表示当向量长度为无穷大时向量处理机的最高性能，也称为峰值性能

$$R_{\infty} = \lim_{n \rightarrow \infty} \frac{\text{向量指令序列中浮点运算次数} \times \text{时钟频率}}{\text{向量指令序列执行所需的时钟周期数}}$$

- 上例中有2条浮点运算指令，MULTFV和ADDV
 - 若处理机时钟频率为200MHz

$$R_{\infty} = \lim_{n \rightarrow \infty} \frac{2 \times n \times 200}{\left\lceil \frac{n}{64} \right\rceil \times 64 + 3n} = \lim_{n \rightarrow \infty} \frac{2 \times n \times 200}{4n} = 100 \text{ MFLOPS}$$

4.4.2 最大性能和半性能向量长度

◆ 2、半性能向量长度 $n_{1/2}$

- 半性能向量长度 $n_{1/2}$ 是指向量处理机的性能为其最大性能 R_{∞} 的一半时所需的向量长度
- 若向量长度 $n=n_{1/2}$ ，则表明整个向量流水线处理时间只有一半是在做有效操作，另一半则是浪费了。

◆ 举例

- 使用上例中的数据，假设时钟周期频率是200MHz，求半性能长度 $n_{1/2}$ ？
- 假设该向量处理机的峰值性能 $R_{\infty}=100$ MFLOPS，一半性能就是 $100/2=50$ MFLOPS，所以根据半性能向量长度的定义有：

4.4.2 最大性能和半性能向量长度

◆ 2、半性能向量长度 $n_{1/2}$

$$\frac{2 \times n_{1/2} \times 200}{\left\lceil \frac{n_{1/2}}{64} \right\rceil \times 64 + 3n_{1/2}} = 50$$

假设 $n_{1/2} \leq 64$ ，那么有：

$$64 + 3n_{1/2} = \frac{2 \times n_{1/2} \times 200}{50} = 8n_{1/2}$$

$$5n_{1/2} = 64, \quad n_{1/2} = 12.8$$

$$n_{1/2} = 13$$

4.4.3 向量长度临界值

◆ 向量长度临界值 n_v

- 向量长度临界值 n_v 是指：对于某一计算任务而言，向量方式的处理速度优于标量串行方式处理速度时所需的最小向量长度。

◆ 对于上述DAXPY的例子

- 假设，在标量串行工作方式下实现DAXPY循环的开销为10个时钟周期。
- 那么在标量串行方式下，计算DAXPY循环所需要的时钟周期数为：

$$T_s = (10 + 12 + 12 + 7 + 6 + 12) \times n_v = 59n_v$$

4.4.3 向量长度临界值

◆ 向量长度临界值 n_v

- 在向量方式下，计算DAXPY循环所需要的时钟周期数为：

$$T_v = 64 + 3n_v$$

- 根据向量长度临界值的定义，有：

$$T_v = T_s$$

$$64 + 3n_v = 59n_v$$

$$n_v = \left\lceil \frac{64}{56} \right\rceil = 2$$

4.5 向量处理机实例

系统型号	推出时间	最大配置, 时钟周期, 操作系统/编译系统	特色和要点
Cray 1S	1976年	有10条流水线的单处理机, 12.5ns, COS/CF7 2.1	第一台基于ECL的超级计算机
Cray 2S/4-256	1985年	256M字存储器的4台处理机, 4.1ns, COS或UNIX/CF77 3.0	16K字的本地存储器, 移植了UNIX V
Cray X-MP 416	1983年	16M字存储器的4台处理机, 128M字SSD, 8.5ns, COS/CF77 5.0	使用共享寄存器组用于IPC
Cray Y-MP 832	1988年	128M字存储器的8台处理机, 6ns, CF77 5.0	X-MP的改进型
Cray Y-MP C-90	1991年	每台处理机2条向量流水线, 16台处理机, 4.2ns, UNICOS/CF77 5.0	最大的Cray机器
CDC Cyber 205	1982年	有4条流水线的单处理机, 20ns, 虚拟OS/FTN200	存储器-存储器系统结构

4.5 向量处理机实例

系统型号	推出时间	最大配置, 时钟周期, 操作系统/编译系统	特色和要点
ETA 10E	1985年	单处理机, 10.5ns, ETAV/FTN 200	Cyber 205的后继型号
NEC SX-X/44	1991年	每台处理机4组流水线, 4台 处理机, 2.9ns, F77SX	
Fujitsu VP2600/10	1991年	5条流水线的单处理机和双标 量处理机, 3.2ns, MSP.EX/F77 EX/VP	使用可重构微向 量寄存器和屏蔽
Hitachi 820/80	1988年	512MB存储器, 18个流水线 功能部件的单处理机, 4ns, FORT 77/HAP V23-OC	64个I/O通道, 最大传输率为 288MB/秒